



Sunbeam Infotech

Exploring new ideas, Reaching new heights!



OOP

→ major

→ abstraction

→ encapsulation

→ hierarchy → inheritance, association

→ modularity

→ minor

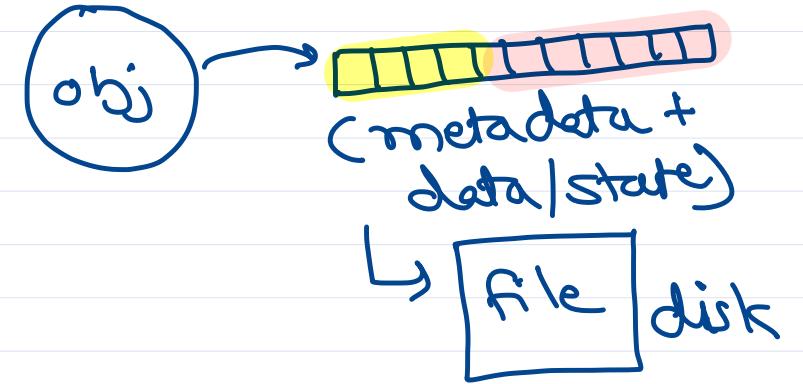
→ typing → polymorphism

→ concurrency

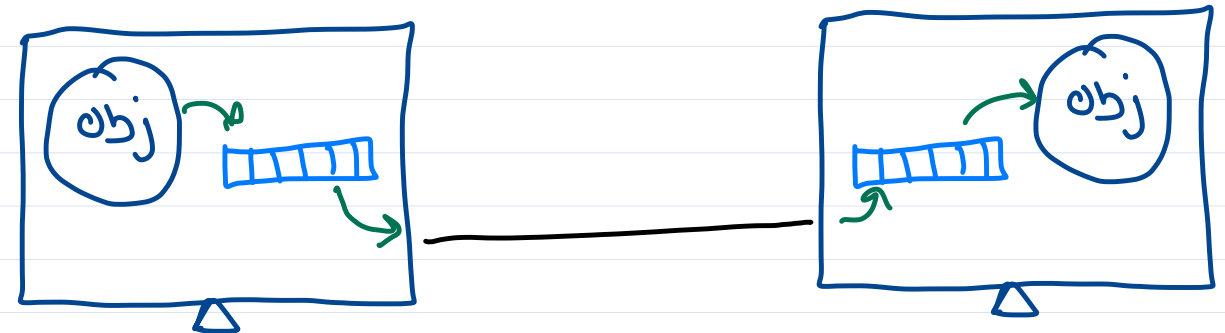
→ persistence

Persistence

① serialization



② marshalling



Data security

```
class Time {
```

```
    private:
```

```
        int h, m, s;
```

```
    public:
```

```
        // Setter
```

```
        void inc(int sec);
```

```
    };
```

```
}
```

```
Time t1 (23, 59, 59);
```

```
t1.inc(2);
```

```
t1 → 0:0:1
```

```
Time t2;
```

```
t2.h = 49; X
```

```
t2.m = 95; X
```

```
t2.s = 105; X
```

```
t2.inc(92);
```

```
t2.seth(49);
```

```
t2.setm(95);
```

```
t2.sets(105);
```

check/validation

Setter

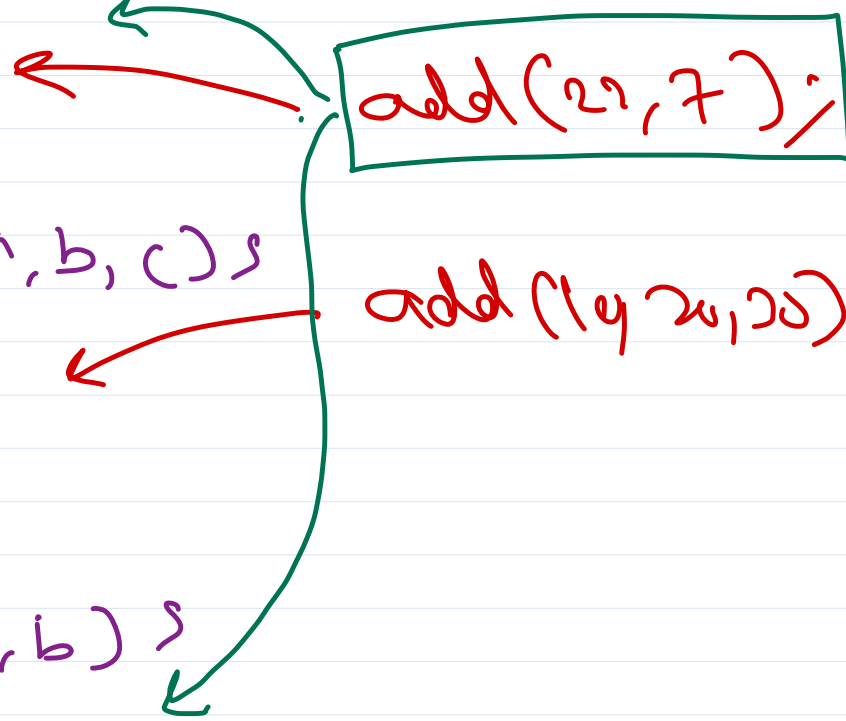


Overloading vs Overriding

```
int  
add(a, b) {  
    =  
}  
  
add(a, b, c) {  
    =  
}  
  
float  
add(a, b) {  
    =  
}
```

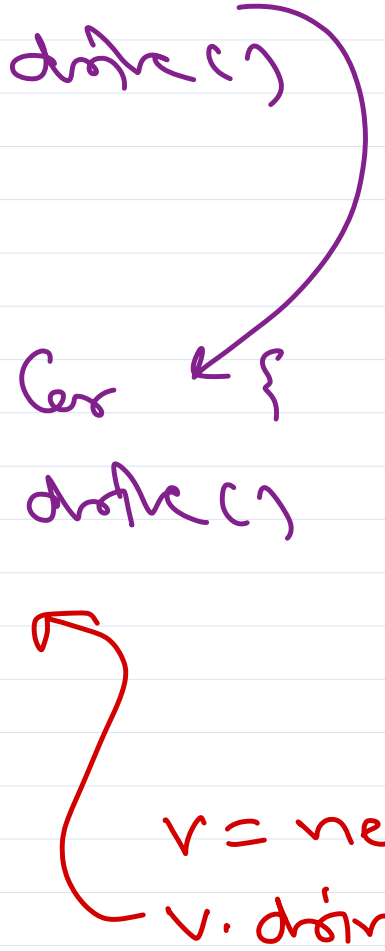
`add(2, 7);`

`add(1, 2, 3);`



```
class Vehicle {  
    void drive() {  
    }  
}  
  
class Car {  
    void drive() {  
    }  
}
```

`v = new Car();`
`v.drive();`



Hierarchy

hierarchy

① is-a → Inheritance

✓ User ← [Customer
Employee

② has-a → Association

✓ Employee → Joining Date

③ use-a → dependency

✓ Service → Dao

✓ Controller → Service

④ creates-a → instantiation/factory → Product Factory creates Product.

✓ Session Factory → Session

(spl)

(gen)

→ Apple is-a Fruit

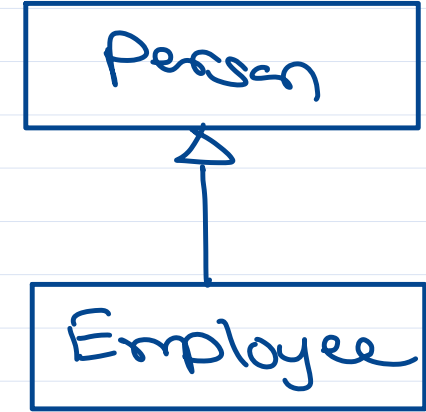
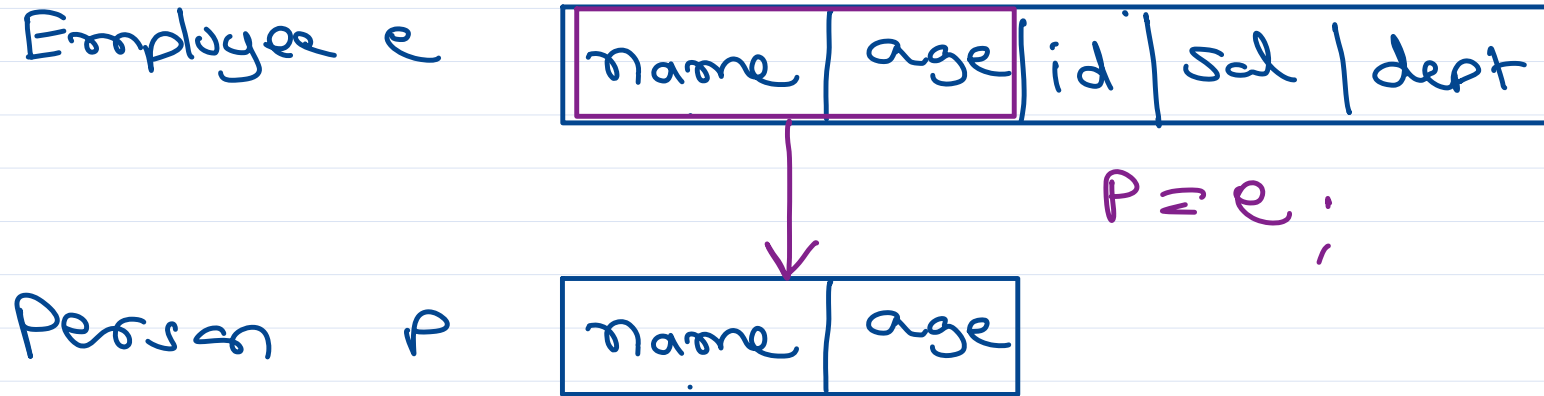
→ Computer has-a Motherboard

→ Document depend on Printer.

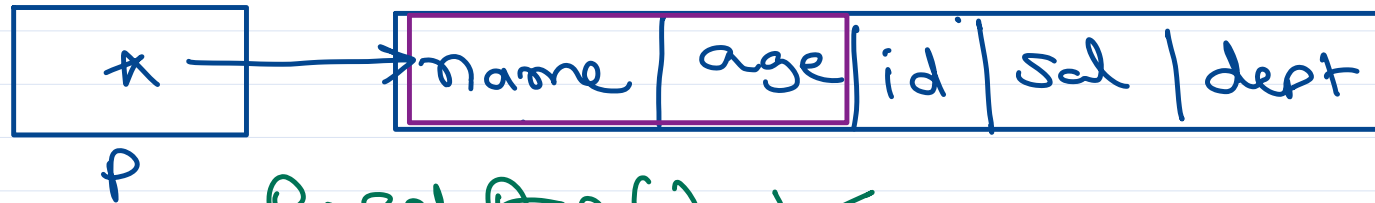
association → Composition (tight)
→ aggregation (loose)



Object slicing



```
Person p = new Employee();
```



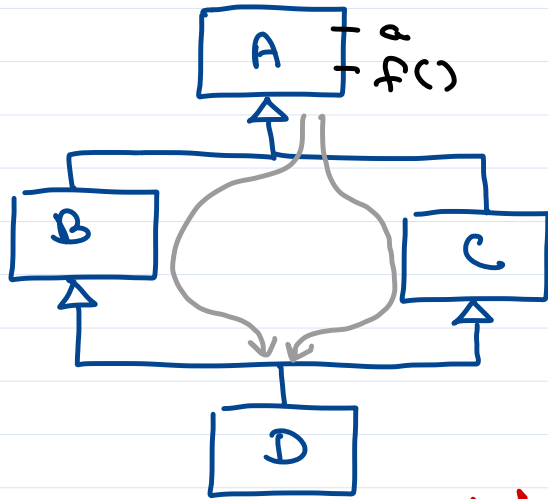
$p.setAge()$ ✓

$p.setSal()$ ✗

$p.display()$ ✓ overridden.

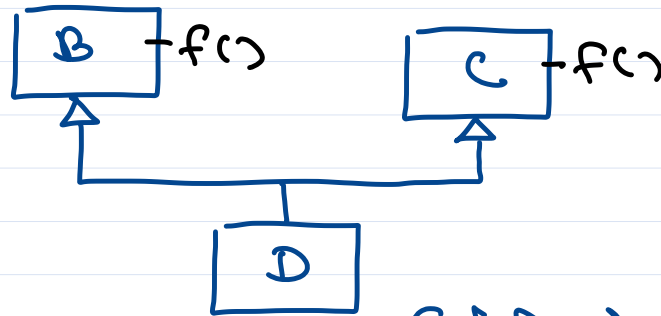


Diamond problem and Multiple inheritance



C++
D obj;
obj.a = 10; X
obj.f(); X

Solution: inherit A
virtually into B & C
so that single copy of
A members is inherited
to D.



D obj;
obj.f();

C++ → B & C classes f() { }
↳ obj.B::f();

Java 8 → B & C interfaces
f() { } default method
↳ soln → override f() in D.



Serialisation

class MyClass implements Serializable

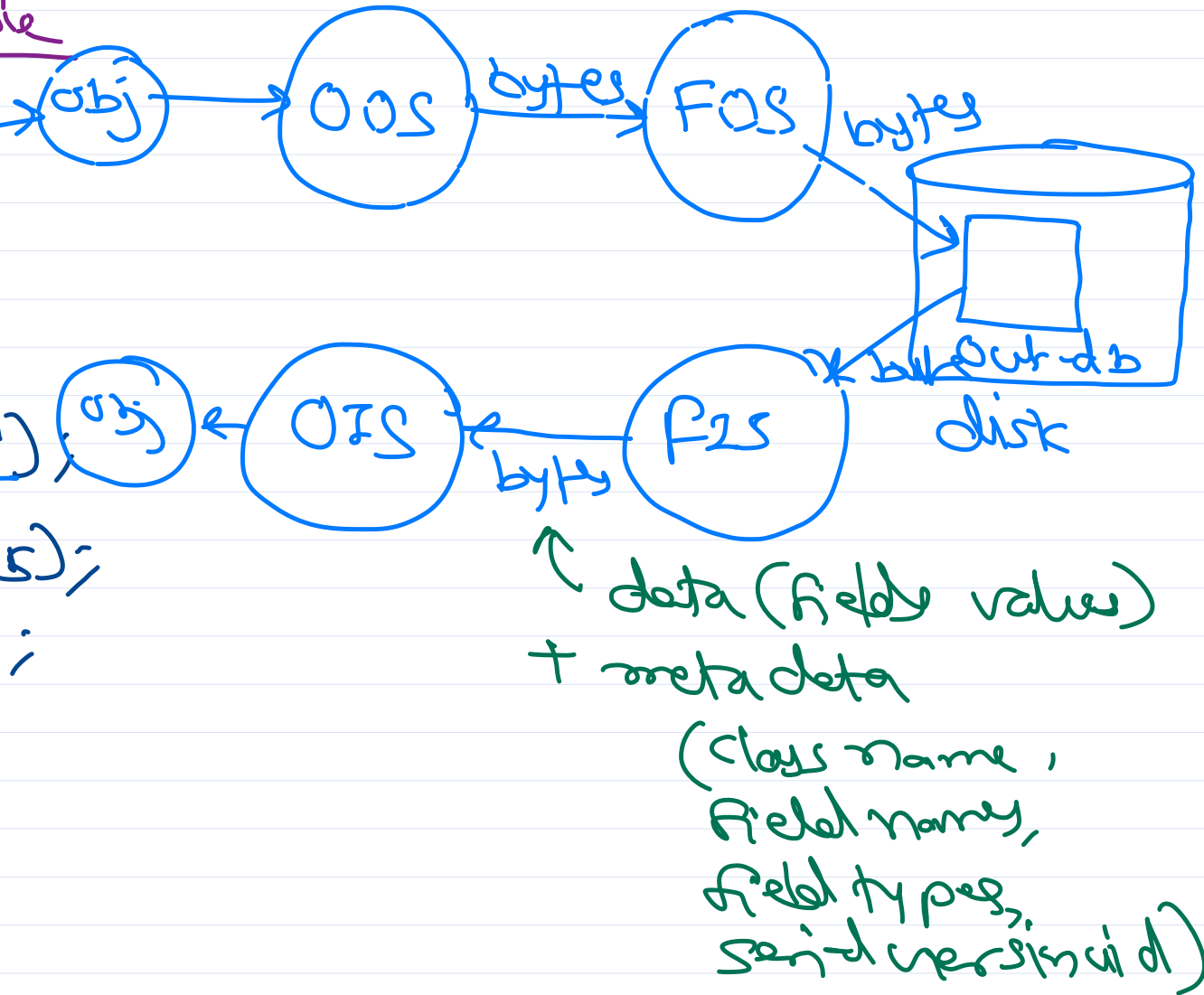
?

{
 ...
}

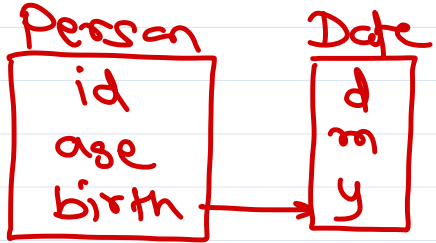
MyClass obj = new MyClass();

- FOS fos = new FOS("out.db");

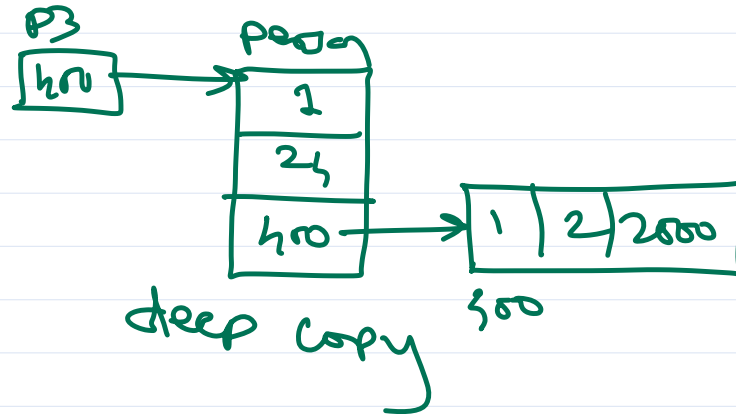
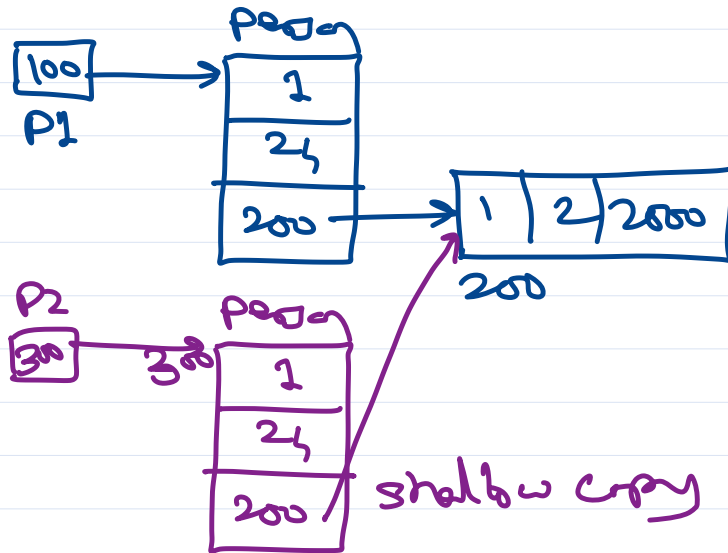
- OOS oos = new OOS(fos);
oos.writeObject(obj);



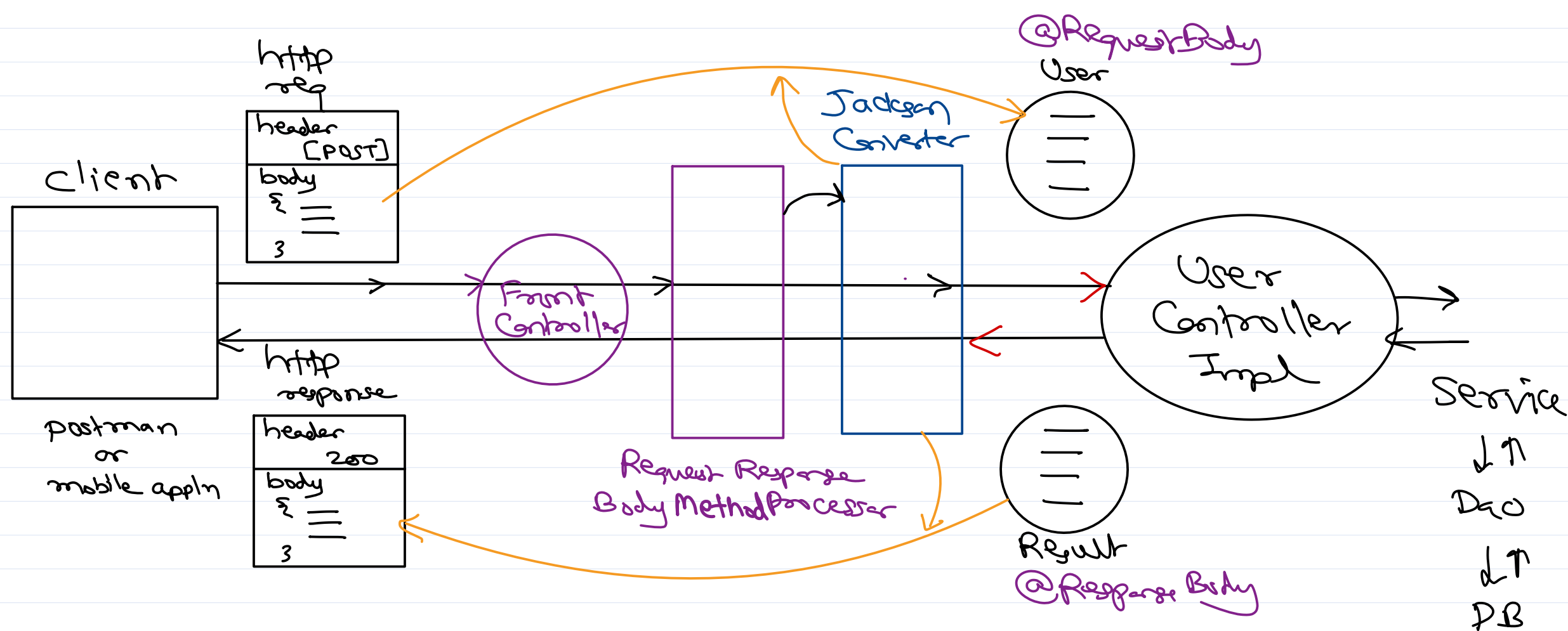
Deep vs Shallow copy



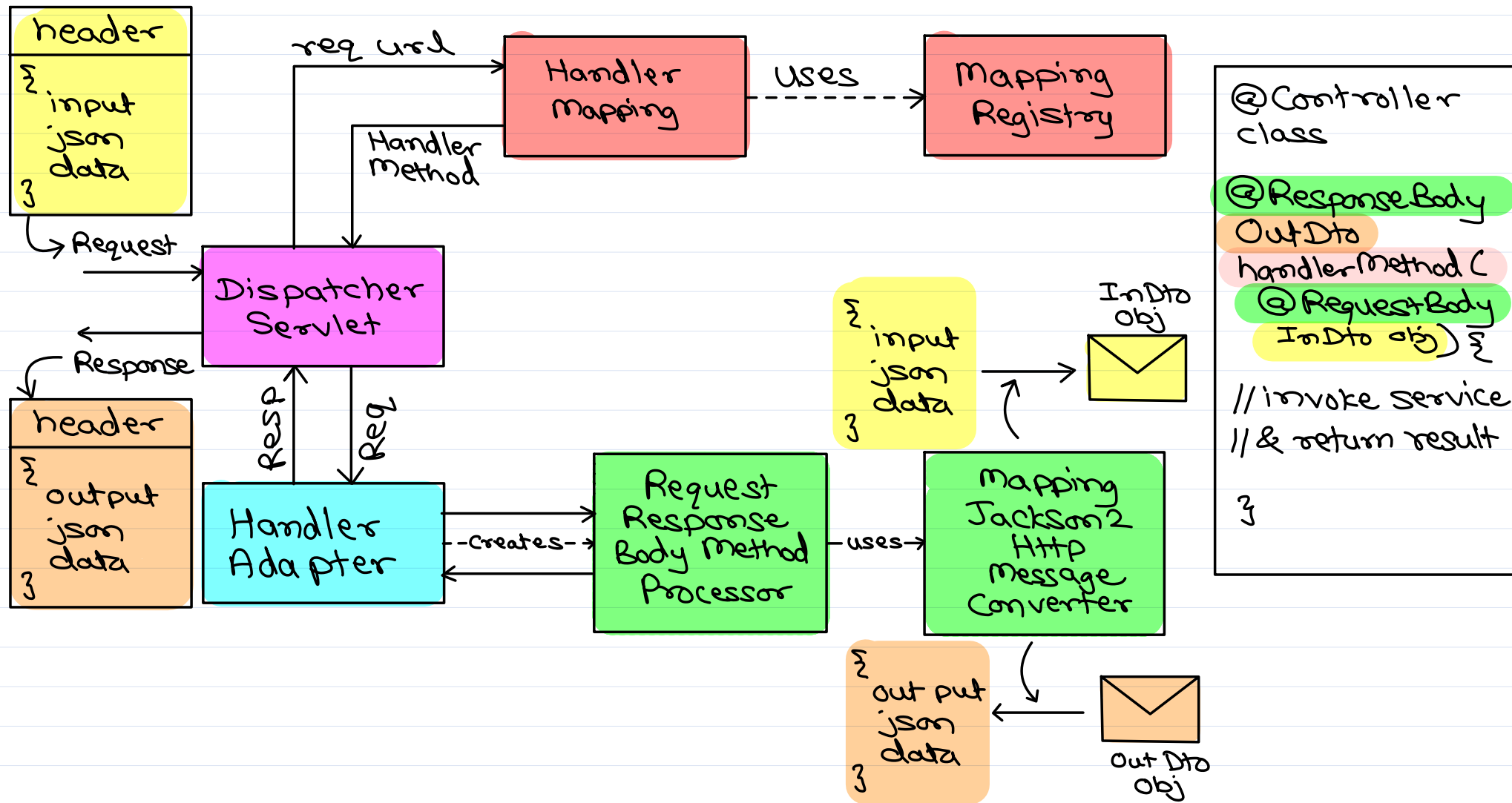
`p1 = new Person(1, 24, new Date(1, 2, 2000))`



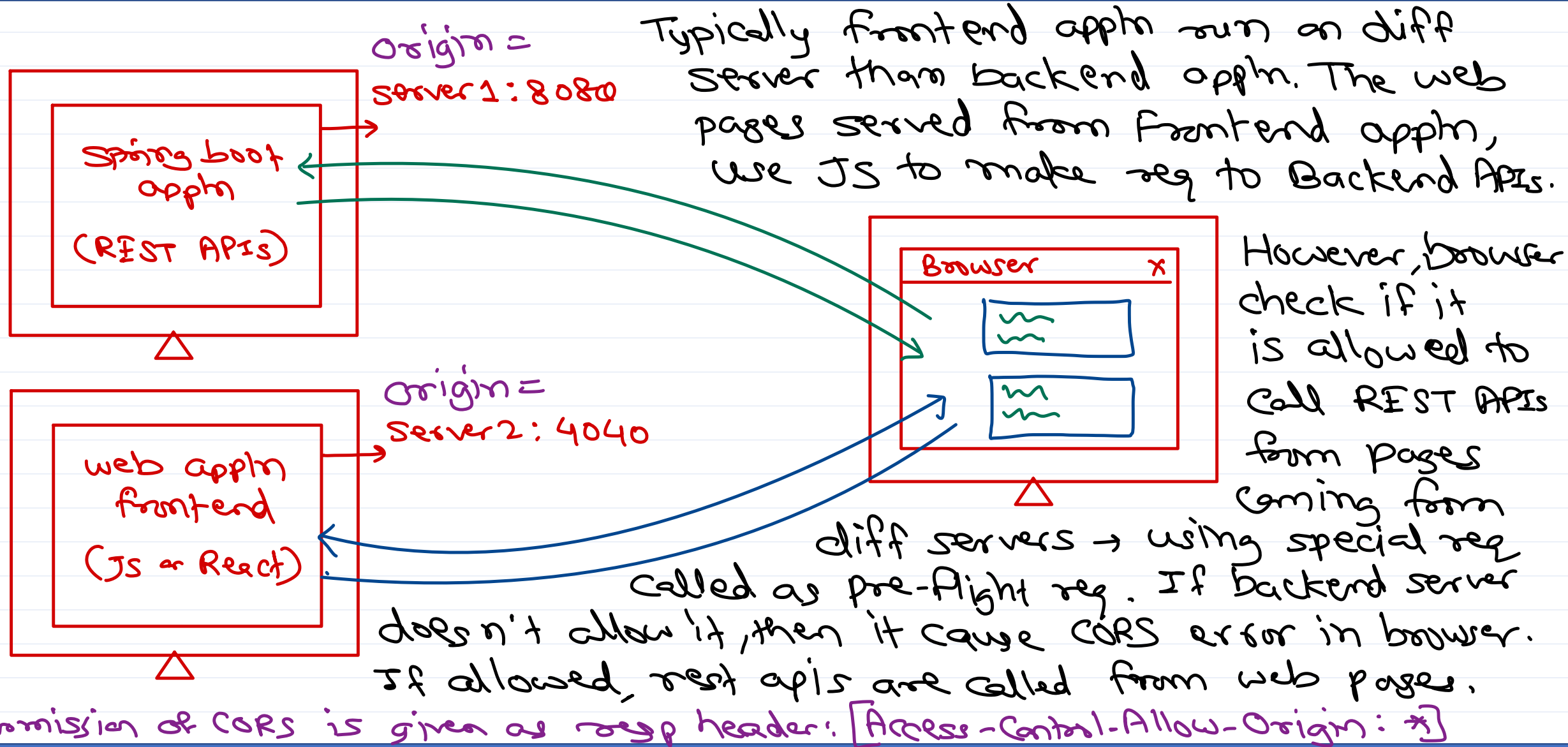
REST Services



REST Services



CORS



JWT

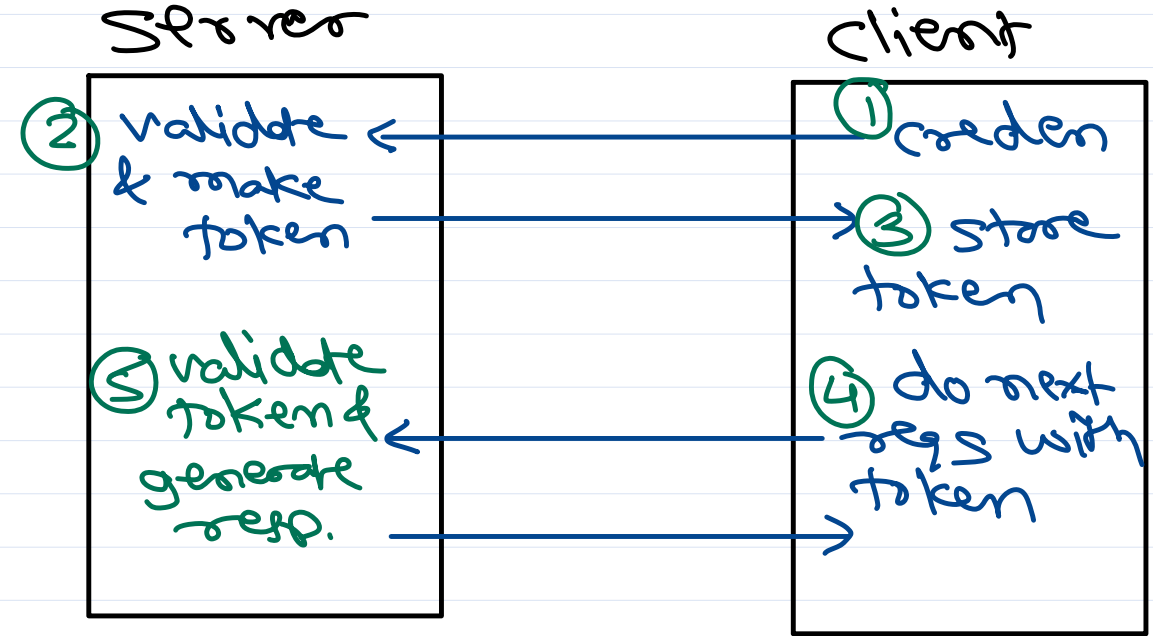
- * Used for REST api authentication.
- * REST apis are stateless.
 - Each req should include all input details that it needs (cannot rely on prev request inputs - not stored).
 - Each req should have user identity.

* JWT token is used to store user identity. It can be carried with each request (like cookies).

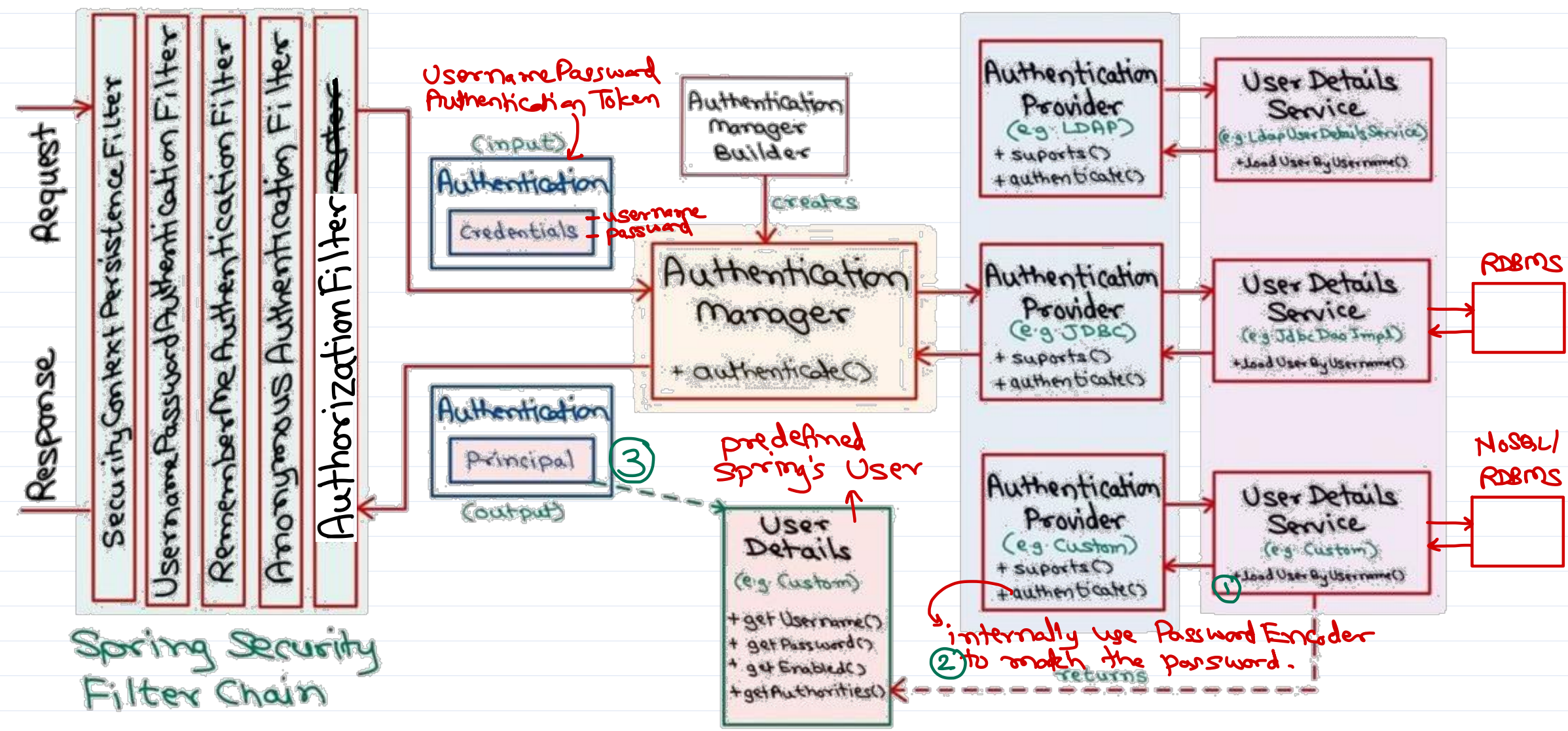
* JWT token has 3 parts

(A) Header (B) Payload (data) (C) Signature
 ↓ ↓ ↓
 enc algo data to store secret key.
 in token (id)

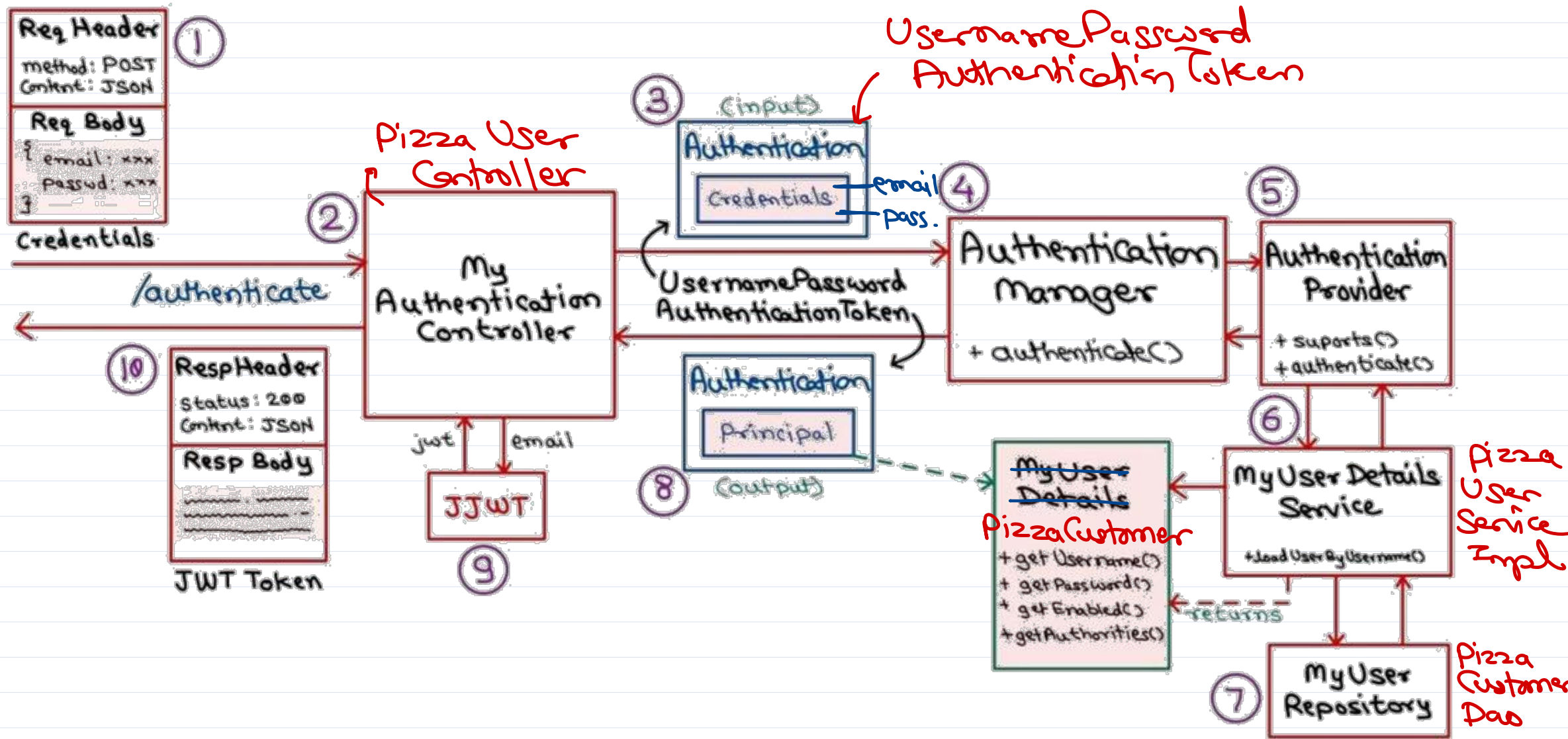
* JWT can be read by client, but can't be tampered/modified.



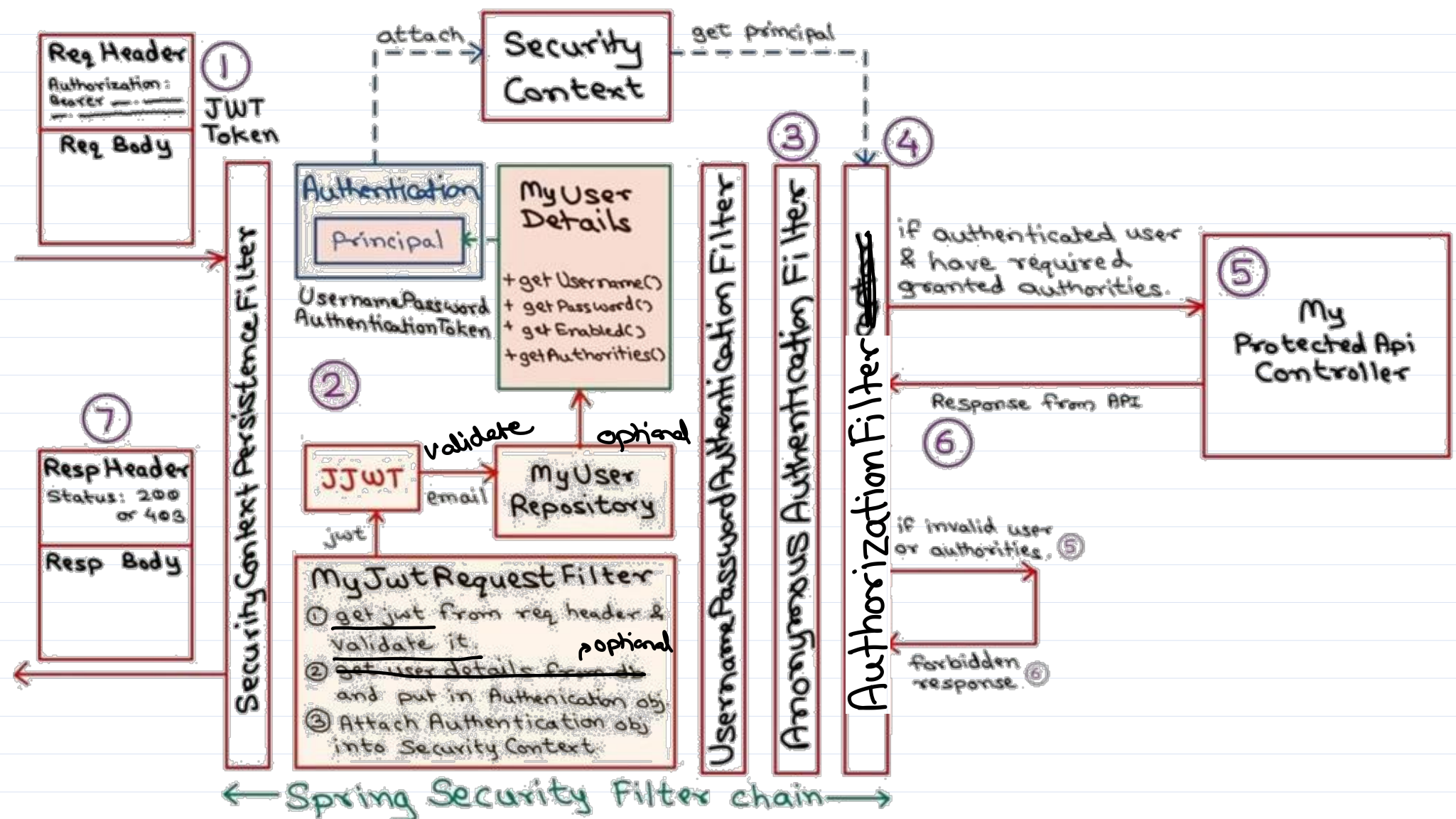
Spring Web Security



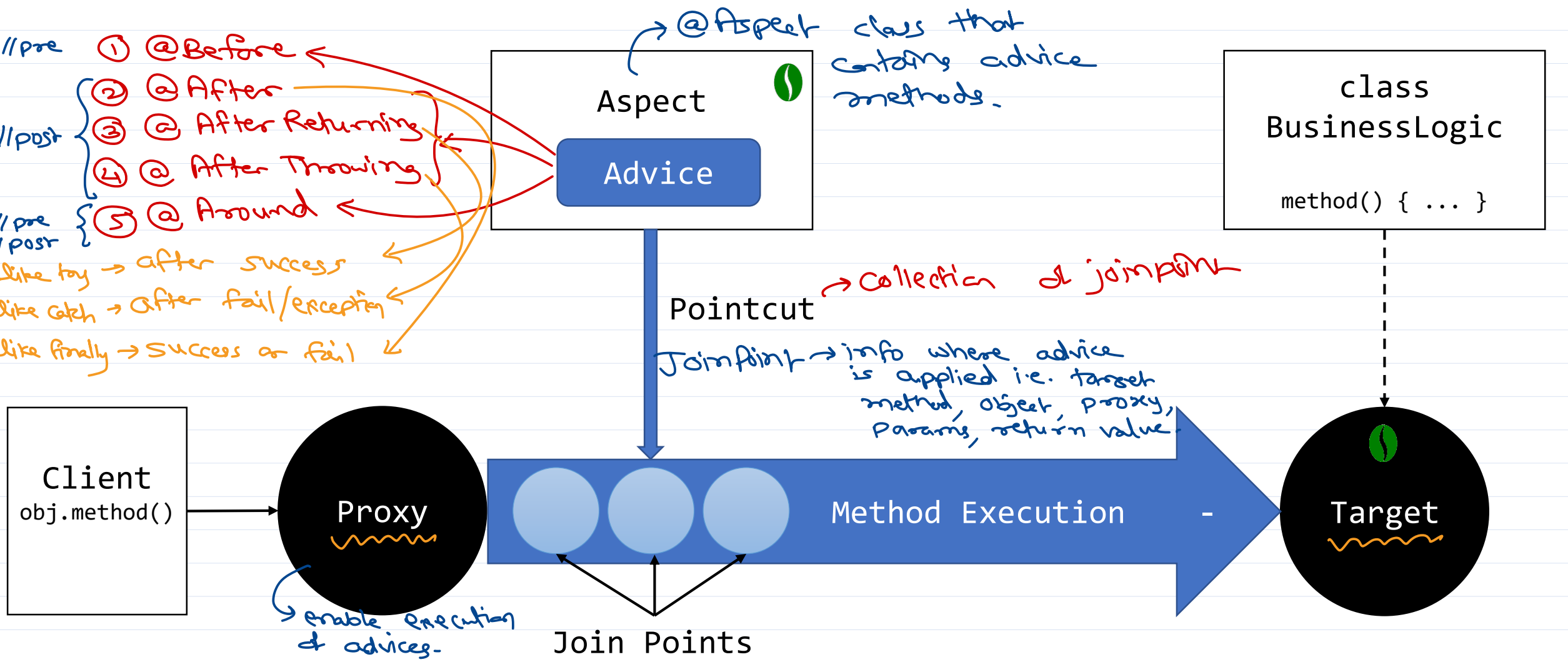
Spring REST Security (1)



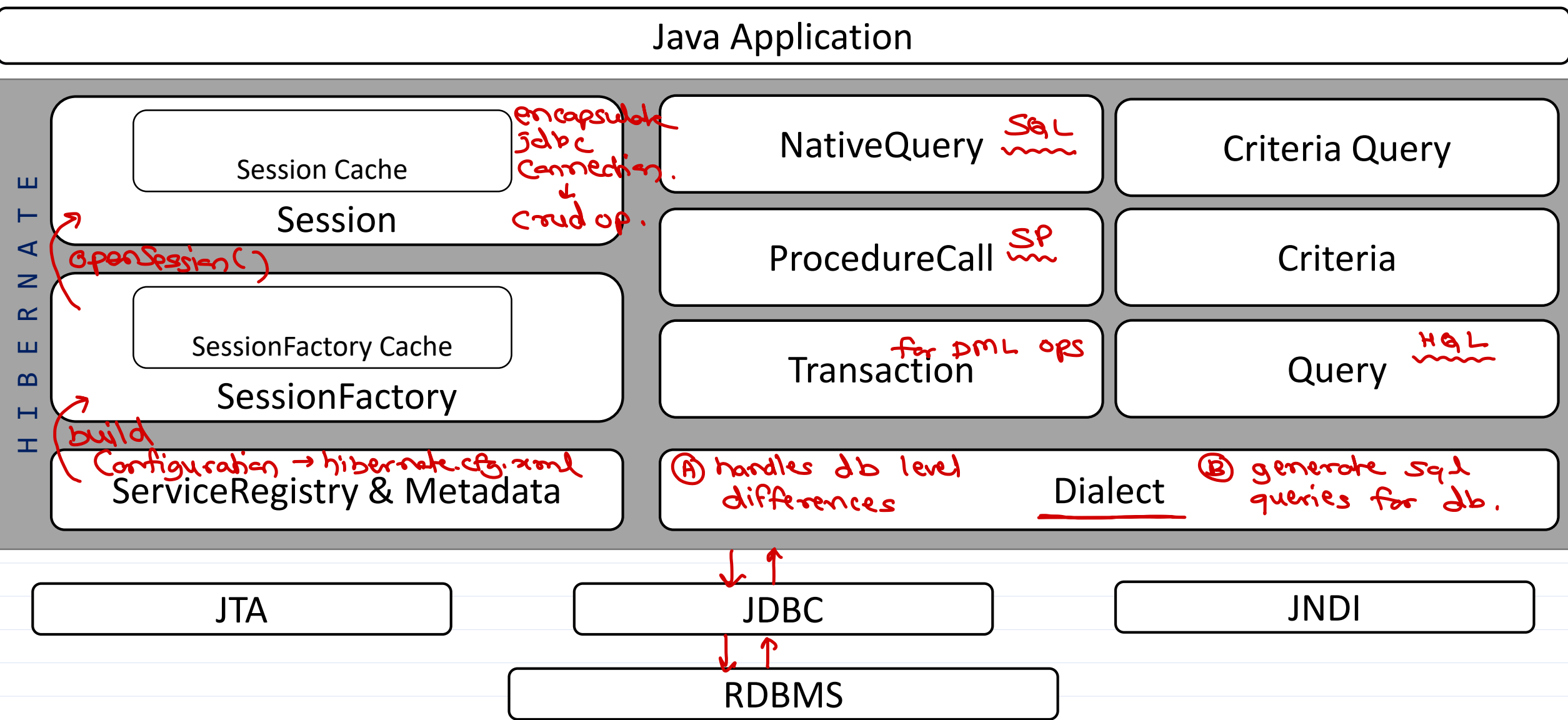
Spring REST Security (2)



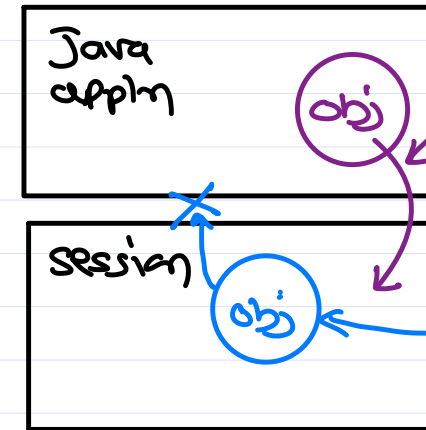
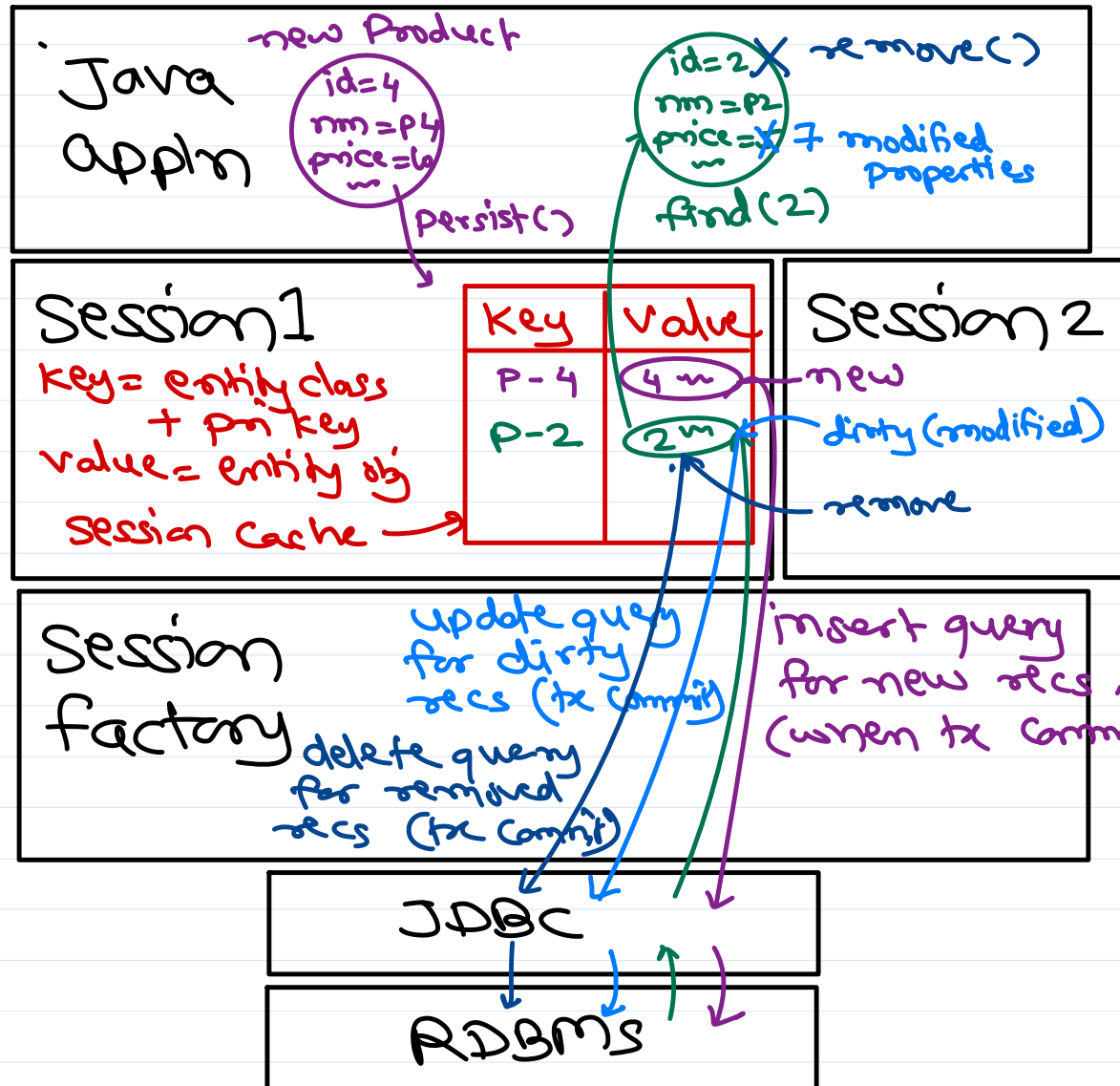
Spring AOP



Hibernate Architecture



Hibernate Caching



merge() - add obj into session cache & mark it as new (if not present in db) or dirty (if present in db).

detach() - remove obj from session cache.

* **Session Cache a.k.a. L1 Cache:**

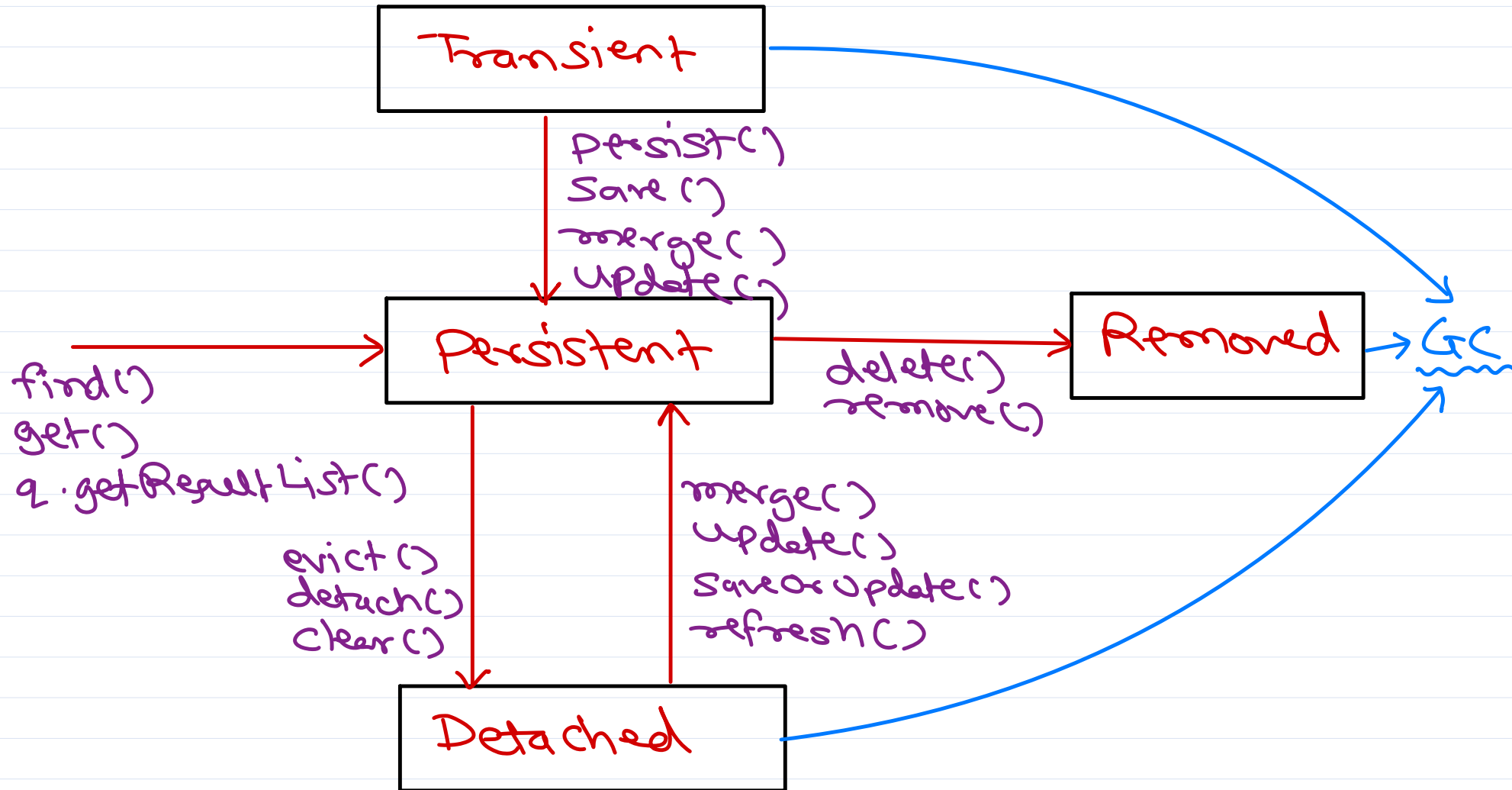
- ✓ always enabled (cannot be disabled)
- ✓ if multiple sessions refer same row in db, they will create separate objs.

* **Session factory Cache a.k.a. L2 Cache.**

- ✓ by default disabled, - need to configure.
- ✓ stores entity objs in serialized form.
- ✓ Common cache across all sessions i.e. multiple sessions get data from L2 cache & create java obj in their cache.
- ✓ speed up execution - less db connectivity.

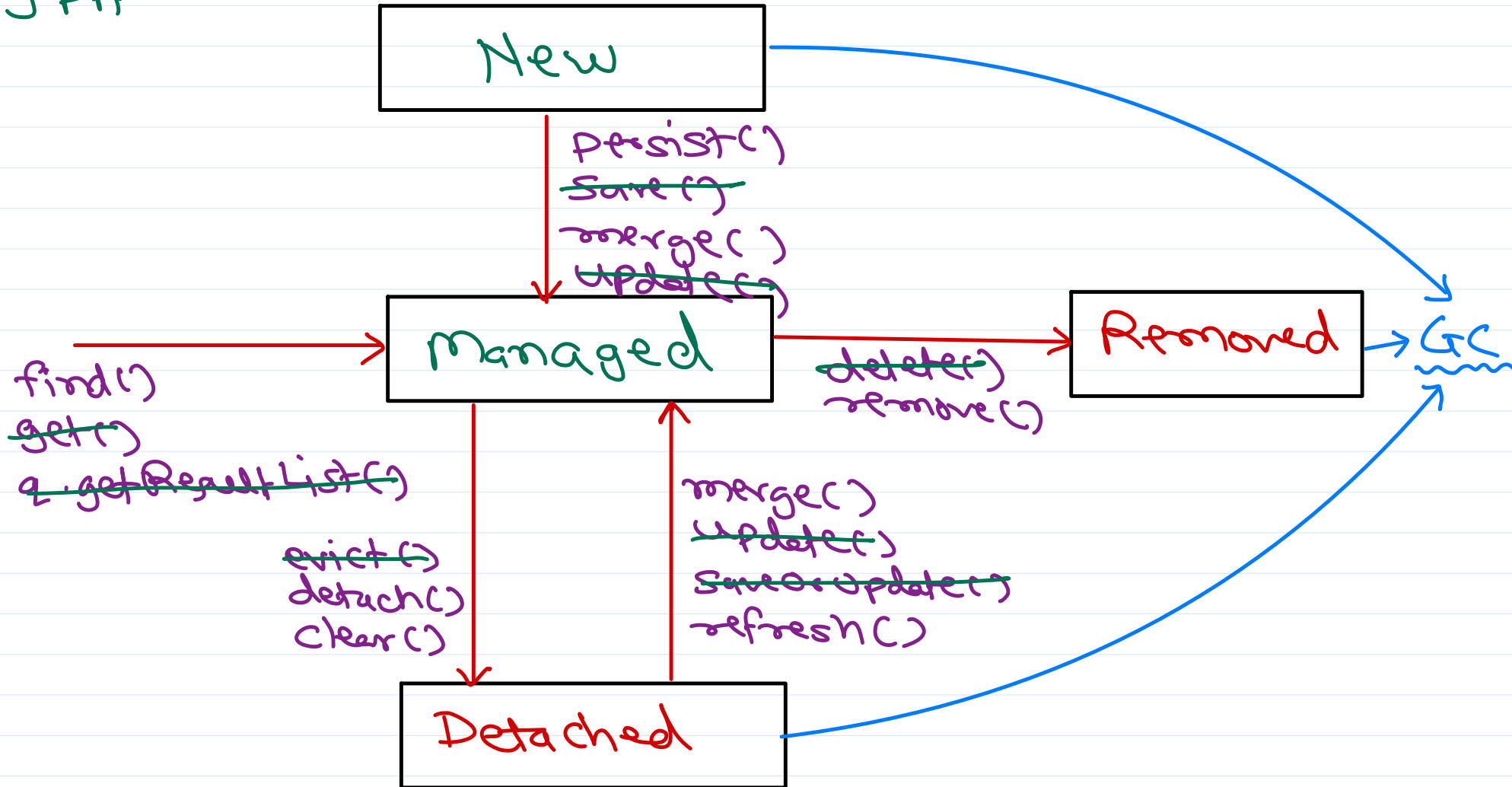


Hibernate Entity Life Cycle



~~Hibernate~~ Entity Life Cycle

JPA







Thank you!

Nilesh Ghule <nilesh@sunbeaminfo.com>

